

PKCERT AI & Software Development Internship

Task 28: Production LLM Serving, Asynchronous Microservices & Full-Stack Deployment

Abdullah Amir

AI and Software Development
24 August 2026

Full code lives in this task's directory (`app/`, `frontend/`, `Dockerfile`, `docker-compose.yml`, `benchmark/`). An asynchronous FastAPI microservice serves three Hugging Face `transformers` pipelines (sentiment, summarization, generation), containerized via a multi-stage Docker build, orchestrated locally via `docker-compose`, and paired with a self-contained HTML/CSS/JS frontend served from the same container – deployment-ready for Hugging Face Spaces (Docker SDK), whose free CPU tier's 16GB RAM comfortably fits this task's three-model footprint, unlike Render/Railway's ~512MB–1GB free tiers.

Part A: Pretrained LLM Inference & Multi-Task Orchestration

A1 – models, and why.

Task	Model	Params	Rationale
Sentiment	cardiffnlp/twitter-roberta-base-sentiment-latest	125M	Multi-class (neg/neu/pos), not the more common binary SST-2 default – the brief specifically asks for multi-class.
Summarization	t5-small	60M	Deliberately the lightest viable abstractive summarizer, not a BART-family model (306M-400M) – a direct instance of A4's memory-footprint trade-off.
Generation	distilgpt2	82M	The standard lightweight causal LM for CPU-only serving.

Combined resident footprint once all three are loaded: **~695MB RSS** (measured directly, D2), comfortably inside HF Spaces' free 16GB tier.

A2 – decoding strategies, compared with real output. All five are implemented and selectable per-request via `decoding_strategy`. Prompt: *"The best way to learn programming is"*, `max_new_tokens=30`:

Strategy	Sample output (truncated)	Diversity	Coherence
Greedy	"...to learn the basics of programming.\n\n..."	None	Degenerates into repetition
Beam (4)	"...to learn the basics of programming.\n\n..."	Low	Same repetition failure mode
Top-k (50)	"...to take time off of school to become a better, happier, and smarter person."	Moderate	Good
Top-p (0.9)	"...to develop your own languages. We've seen how programming languages do..."	High	Good
Temperature (1.0)	"...to set aside your mind to do so. The best way to teach this is..."	Highest	Lower

Greedy and beam search both degenerated into repetition on this open-ended prompt – a real, observed failure mode, not just textbook description: both always pick the (locally or globally) highest-probability continuation, which for free-form generation tends toward short repeating loops once the model enters one. Top-k, top-p, and temperature sampling all avoided this by introducing randomness into token selection, at the cost of losing greedy’s determinism. **Latency impact:** decoding strategy affects output quality far more than per-token latency at this model size – all five perform one forward pass per generated token; beam search is the one real exception, since it evaluates `num_beams` candidate sequences per step rather than one.

A3 – payload sanitization & context-window management. Input is stripped of non-printable characters, tokenized with the *target model’s own tokenizer* (word count and subword-token count diverge – Task 27’s finding), and truncated at the token level (never mid-token) when it exceeds a per-task limit, rather than erroring outright. **A genuine bug this surfaced:** naively truncating token ids, decoding back to text, and handing that text to the pipeline is not token-count-lossless – the pipeline re-tokenizes the decoded text from scratch and adds fresh special tokens, which can push the count back over a model’s true limit. This produced a real `IndexError: index out of range in self` from RoBERTa’s position-embedding lookup on an 800-word test input. Fixed with a bounded re-check-and-re-trim loop (D3 has the full account); re-verified afterward with the same input, now truncating cleanly to 511 tokens with no error.

A4 – architectural trade-off: local weights vs. remote inference API.

Dimension	Direct weight loading (this task)	Remote inference API
Memory	Full model resident in-process (~695MB, all three)	Zero local model memory
Compute	Every request pays full CPU forward-pass cost	Runs on provider’s (often GPU) infra
Latency	No extra network hop; 24–500ms once warm	Extra round trip, but often GPU-fast
Cold start	Measured, three tiers: 171s (down-load), 2.2–6.9s (cache-warm), sub-second (fully warm)	Opaque, provider-controlled
Rate limits	None – bounded only by this host’s CPU/RAM	Provider-imposed, often stricter on free tiers

Direct loading wins here specifically because the three models are small (267M params combined) – a 70B-parameter model would make direct CPU loading impractical on memory alone, at which point a remote GPU-backed API becomes the only viable option regardless of its network/rate-limit costs.

Part B: Asynchronous REST API Engineering

B1 – endpoints. `POST /api/v1/sentiment, /summarize, /generate, GET /healthz`.

B2 – validation & error handling. Pydantic v2 models enforce type/length/range bounds (`temperature` ∈ [0.01, 2.0], `top_p` ∈ [0, 1], `max_new_tokens` ∈ [1, 256]) *before* any request reaches inference code – verified directly: an out-of-range temperature (5.0) returned 422 with a structured body before touching a model. Dedicated handlers map `ContextWindowError`→400, `RequestValidationError`→422, and any unhandled exception→500, all sharing one `{error, detail, status_code}` response shape.

B3 – concurrency & state. Every Hugging Face pipeline call is synchronous, CPU-bound Python – calling it directly inside an `async def` route would still block the single event loop for

the full inference duration. Every inference call is dispatched via `run_in_threadpool` instead, keeping the event loop free to route other requests (including `/healthz`) while CPU-bound work runs on a worker thread. `get_pipeline()` is a lock-guarded lazy-loading singleton: the first request for a task pays its load cost once; every later request (any caller) reuses the cached pipeline. `PRELOAD_MODELS=true` (the Docker default) instead pays all three models' cost once at startup – A4's cold-start table is the direct evidence behind this trade-off. Structured JSON request logging and CORS middleware are both configured.

Part C: Containerization & Cloud Infrastructure Deployment

C1 – multi-stage Dockerfile. Stage builder (python:3.11-slim) resolves and installs dependencies – including the heavy `torch/transformers` wheels – into an isolated virtualenv, in its own layer; since `requirements.txt` is copied and installed *before* application code, editing `app/*.py` never invalidates this layer, so a code-only rebuild skips re-downloading torch entirely. Stage runtime starts from a fresh minimal base and copies over only the built virtualenv + source – none of the builder stage's pip cache or intermediate layers survive into the final image. Runs as a dedicated non-root user; includes a container-level `HEALTHCHECK`. **Measured final image size: 1.73GB** – critically, this required explicitly pinning `pip install torch --index-url https://download.pytorch.org/whl/cpu` *before* the general `requirements.txt` install; a plain `pip install torch` instead pulls PyPI's default CUDA-enabled build, bundling multi-hundred-MB NVIDIA driver libraries this CPU-only deployment target never uses – caught by watching the first build's download log show an unexpectedly large torch wheel mid-build, not assumed in advance. **Verified end to end, not just built:** `docker run` on the resulting image loaded all three models (matching A4's cold-start timings), `docker exec ... whoami` confirmed the container runs as the non-root `app` user, Docker's own `HEALTHCHECK` reported `healthy`, and all three API endpoints plus the static frontend responded correctly against the live container.

C2 – local orchestration. `docker-compose.yml` configures environment variable injection (`PORT`, `PRELOAD_MODELS`, optional `.env` for `HF_TOKEN`), an automated health check (240s start period, since preloading pays all three models' cost before reporting healthy), host port 8000 mapped to container port 7860, a named volume persisting the Hugging Face model cache across restarts, and explicit CPU (2.0)/RAM (2GB limit, 1GB reservation) quotas.

C3 – cloud deployment. Target: Hugging Face Spaces (Docker SDK) – chosen over Render/Railway specifically because its free CPU tier offers 16GB RAM against their ~512MB–1GB, comfortably fitting this task's 695MB three-model footprint without needing to drop a model. A GitHub Actions workflow (`.github/workflows/deploy-hf-spaces.yml`) mirrors this directory to a Space's git remote on every push to `main`, which triggers Spaces' own automatic rebuild/redeploy – the CI/CD pipeline the brief asks for. Actually pushing this live requires an account-specific Hugging Face token, which was left as the final, credential-gated step rather than attempted without the necessary account access (see `README.md`'s step-by-step deployment instructions).

Part D: Frontend, Load Testing & Documentation

D1 – frontend. Plain HTML/CSS/JS, no framework, served by FastAPI itself from the same container – one deployable unit. Three tabs, live parameter controls for generation (the strategy dropdown shows/hides only the relevant sliders), a polling health badge, and per-request status/error reporting. Every request uses relative URLs, so the identical static files work unmodified against localhost, a local container, or the live Spaces URL. Verified in an actual browser (headless Chromium screenshot) during development – the health badge correctly

reported "API online · 44ms · 3 model(s) loaded" once all three pipelines were warm.

D2 – load testing. Single-client (sequential) vs. 6-way concurrent, against the local instance with all three models pre-warmed:

Endpoint	Pattern	P50	P90	P99	Throughput
sentiment	sequential	24.9ms	28.6ms	37.0ms	34.3 req/s
sentiment	6 concurrent	81.2ms	84.3ms	86.1ms	69.5 req/s
summarize	sequential	491.8ms	520.5ms	541.8ms	2.02 req/s
summarize	6 concurrent	3997.8ms	4058.6ms	4079.3ms	1.54 req/s
generate	sequential	308.6ms	343.1ms	352.0ms	3.20 req/s
generate	6 concurrent	1467.8ms	1524.9ms	1527.7ms	4.07 req/s

Peak RSS: 397MB (sentiment load), 399MB (summarize load), 686MB (generate load, all three models resident by this point). Full per-run detail in `benchmark/results/local.json`. This machine had real background load from other running applications during measurement, so these are *this-environment* numbers, not clean-room absolutes – reported transparently rather than presented as idealized.

The real finding: sentiment (the cheapest model) scales throughput well under concurrency (34→69 req/s) with latency growing only ~3x for a 6x concurrency increase. Summarize and generate (the multi-forward-pass generation tasks) show the opposite pattern – throughput barely moves (summarize: 2.02→1.54 req/s, actually *worse*) while per-request latency balloons 5–8x. This is direct evidence of a CPU-bound concurrency bottleneck: `torch` releases Python’s GIL during its C++ tensor ops, so `run_in_threadpool` genuinely allows some overlap, but six simultaneous multi-step generation loops still compete for the same limited CPU cores, and the scheduler dividing those cores six ways serializes the *effective* per-request throughput almost entirely for the heavier tasks.

D3 – two non-trivial engineering challenges.

1. *An unpinned dependency install silently pulled a breaking major version.* `pip install transformers` installed `transformers==5.15.1` (the newest release at build time), which no longer registers "summarization" as a valid pipeline task (`KeyError: Unknown task summarization`, with the printed available-tasks list conspicuously omitting both `summarization` and `text2text-generation` entirely). Diagnosed by reading the actual exception’s task list rather than assuming a typo in this codebase; resolved by pinning to `transformers==4.57.6`, restoring the expected pipeline registry – the same class of "an unpinned major-version jump breaks an established API" issue this internship already documented once, in Task 25 (gensim vs. Python 3.14). `requirements.txt` now pins every dependency with `==`, not a floor, specifically so this failure mode is reproducible-and-fixable rather than silently different on the next install.

2. *The concurrency bottleneck measured in D2.* Rather than assume `run_in_threadpool` was sufficient for real concurrency, D2’s load test was run deliberately, and it revealed that it isn’t – not because the async wiring is wrong, but because six CPU-bound generation loops on a host with limited cores are fundamentally compute-bound, and no amount of async/threading changes that. **The concrete fix this motivates, matching the brief’s own suggested mitigations:** (a) *worker-pool adjustment* – running multiple `uvicorn` worker processes (`--workers N`) would let the OS schedule genuinely independent processes across cores instead of Python threads sharing one interpreter, at the cost of $N \times$ the model memory (a real trade-off against A1/A4’s small-model strategy, not a free win); (b) *quantization* – `torch.quantization.quantize_dynamic` on the linear layers would reduce each forward pass’s compute cost directly, lowering both single-request latency and the per-request cost that com-

pounds under concurrent load, without the memory multiplication worker processes require. Neither was applied in this session (documented here as the identified, justified next step rather than implemented against a deadline).

D4 – reflection. Static-vs-dynamic trade-offs recur throughout this task: small models chosen over larger ones for deployability (A1), lazy-loading traded against startup latency (B3/A4), and concurrency itself trading latency for throughput unevenly across the three task types (D2). No single configuration is correct independent of the deployment target – a resource-constrained free tier and a dedicated multi-core production host would reasonably make different choices at every one of these decision points.